OntarioTech
Engineering
& Applied Science

IEEE SSCI 2023
December 5th-8th 2023
Mexico City, Mexico.

A Task Scheduler for Mobile Edge Computing Using Priority-Based Reinforcement Learning

Amin Avan, Farnaz Kheiri, Qusay Mahmoud, Akramul Azim, Masoud Makrehchi, Shahryar Rahnamayan

Introduction:

- Modern IoT applications (entertainment, healthcare, vehicular, industrial) are:
  - bandwidth-intensive
  - computational-intensive
  - data-intensive
  - strict QoS requirements
- IoT devices have limited resources.
  - Need for offloading workload
    - initial solution: cloud computing
- IoT devices & Cloud:
  - modern IoT application generate & consume huge amount of data
    - Network congestion
    - High transmission delays
    - Data security concern
- Edge computing (EC) offers cloud-like services closer to users.
  - Distributed computing & proximity to IoT devices
  - Heterogeneous computational & storage resources

Introduction: (cont)

- Edge computing (EC) offers cloud-like services closer to users.
  - Distributed computing & proximity to IoT devices
    - Base Stations, Macro-cell Stations, Access Points, Femtocells, Roadside Units
  - Heterogeneous computational & storage resources
    - CPU, GPU, DSP, FPGA.
- Task scheduling
  - Assigning tasks to suitable edge servers to ensure timely and effective processing.
- Goal of task scheduling is to optimize
  - resource utilization
  - minimize latency
  - enhance overall system performance
- Task scheduling optimization problem is modeled by
  - Mixed Integer Programming (MIP)
  - Integer Linear Programming (ILP)
  - Mixed Integer Linear Programming (MILP)
  - Mixed Integer Non-Linear Programming (MINLP)
  - Markov Decision Process (MDP)

Related works:

- Heuristic algorithms:
  - COFE [16]:
    - Offloads the tasks on both edge servers & Cloud.
    - Aims to minimize the average makespan.
  - TCA & MPCPU [17]:
    - TCA: aims to assign each task to the nearest edge server.
    - MPCPU: aims to assign each task to the most powerful CPU edge server.
  - CaGTS [19]:
    - Aims to minimize the delay of task response
  - MAMTS [20]:
    - Aims to minimize the average completion time of the tasks.
- Meta-heuristic algorithms:
  - OWPSO [21]:
    - Scheduling the tasks based on opposition-based whale optimization algorithm with particle swarm optimization.
  - IWHO[22]:
    - Optimize task scheduling problem using wild horse optimization
    - Aims to optimize scheduling time, reduce costs, improve QoS & QoE.

Related works:

- AI-based Algorithms:
  - PG-CTS [24]:
    - A deep reinforcement learning (DRL) approach for scheduling tasks in mobile blockchain networks.
    - Uses small-cell base station as an edge server.
  - PGT [25]:
    - A priority-aware greedy approach based on TOPSIS (Technique for Order Preference by Similarity to Ideal Solution).
    - Aims to optimize response time, energy consumption, and execution cost.
  - DRLOSM [26]:
    - Formulate vehicular task scheduling as a Markov decision process.
    - Solve it with the proximal policy optimization algorithm in DRL.
  - DDLO [27]:
    - Task scheduling problem is formulated as a MIP and solved by distributed deep learning.
    - Aims to optimize energy conservation & QoS.
  - DROO [28]:
    - A binary task scheduling technique using DRL.
    - Aims to improve the data processing capacity of low-power networks.

Problem statement:

- Task scheduling in EC
  - heuristic algorithms
    - Complex & time-consuming calculations.
    - Challenging to be compatible with the dynamic environment of edge computing.
    - Local optimal issue.
  - meta-heuristic algorithms
    - Many parameters should be defined.
    - Hard to adjust the parameters.
  - AI-based methods
    - Strong parallel processing capability.
    - Strong distributed storage and learning capability.
    - Long learning time.
    - Huge state-action space for RL agent.

Proposed RL-based scheduling algorithm (PBQ):

- We formulates the task scheduling problem with Markov Decision Process (MDP).
  - Solve it with the Q-Learning algorithm in DRL.
- Inspired by EDF, we prioritize tasks based on their proximity to the deadline.
- When formulating a problem with MDP, we must generate all the possible state-action spaces.
  - Each state is represented by a binary matrix, where each cell indicates the server allocated to a specific task.

$$\begin{array}{c} \\ T_1 \\ T_2 \\ T_3 \\ T_4 \end{array} \begin{array}{c} \begin{array}{cc} S_1 & S_2 \end{array} \\ \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} \end{array}$$

Proposed RL-based scheduling algorithm (PBQ): (cont)

- Acceleration Techniques in PBQ Algorithm:
  - MDP formulation generates all possible state-action spaces for the RL agent to learn and train.
    - Abundance of state-action spaces decrease the speed of the RL agent's learning process and require substantial memory allocation.
    - Removing invalid, repeated, or deletable state spaces would accelerate PBQ.
  - Steps to accelerate PBQ:
    - Consider only "m" (number of edge users) tasks with the earliest deadlines at each step, then generate all possible state-action spaces.
    - Delete state spaces that offload tasks to a single edge server when there are two or more edge servers.
    - Delete state spaces that offload tasks to multiple edge servers simultaneously.

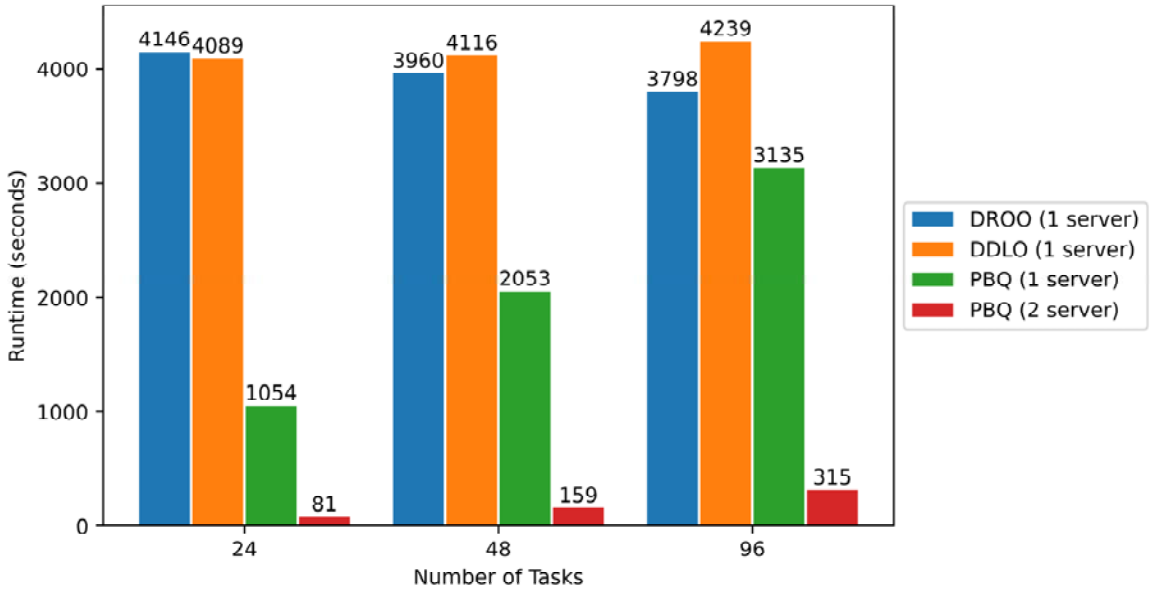
Experimental results:

- We implement PBQ, DDLO [27], and DROO [28] in Python & evaluate their performance using 3 different workloads.
  - Workloads: 24 tasks, 48 tasks, 96 tasks.
    - Each tasks is characterized by required execution time and deadline.
- The experiment is conducted for 31 independent scheduling rounds to verify the validity of our results.
- Performance evaluation metrics are calculated:
  - Runtime
  - Computing Cost
  - RAM Utilization

Experimental results - Runtime:

- Runtime is the time it takes for the algorithm to learn task scheduling.

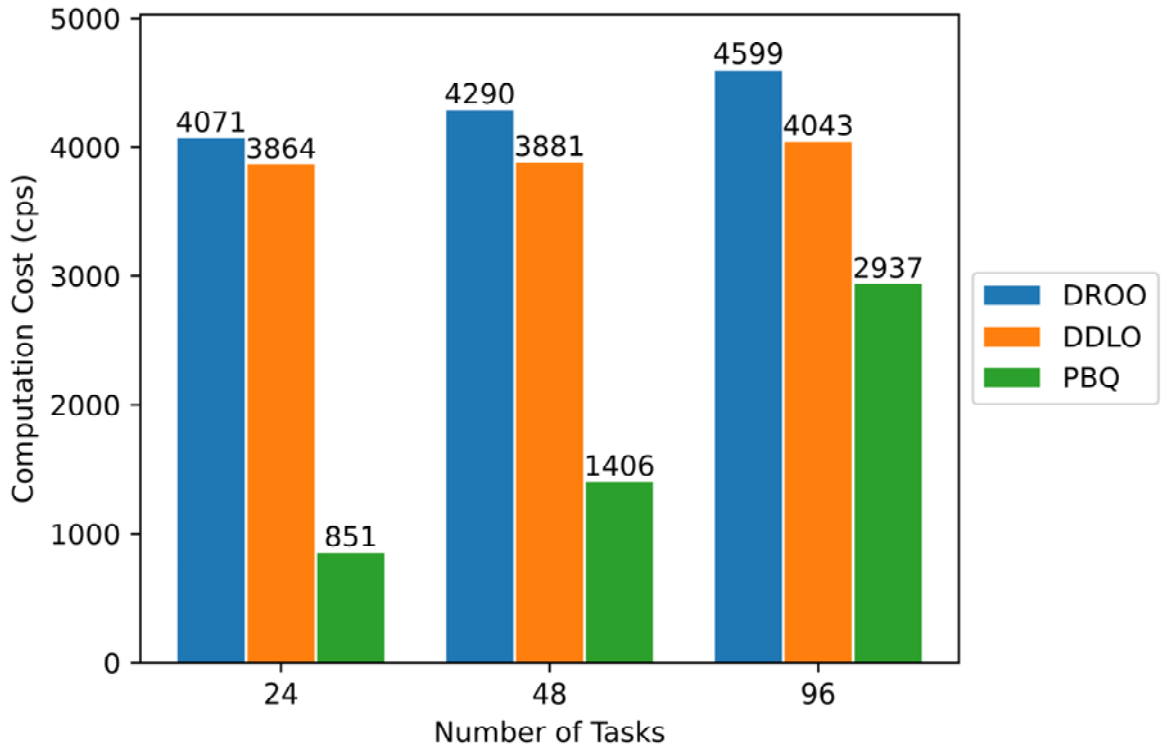
- PBQ is a scalable scheduling algorithm.
- Runtime of PBQ is on average
  77.5% lower than DDLO & DROO in 24 tasks
  54.0% lower than DDLO & DROO in 48 tasks
  21.5% lower than DDLO & DROO in 96 tasks
  when there is only one edge server.
- Runtime of PBQ is on average
  98% lower than DDLO & DROO in 24 tasks
  95% lower than DDLO & DROO in 48 tasks
  91% lower than DDLO & DROO in 96 tasks
  with two edge servers.



Experimental results - Computation cost:

- Computation cost is the number of computation cycles an algorithm requires for its execution.

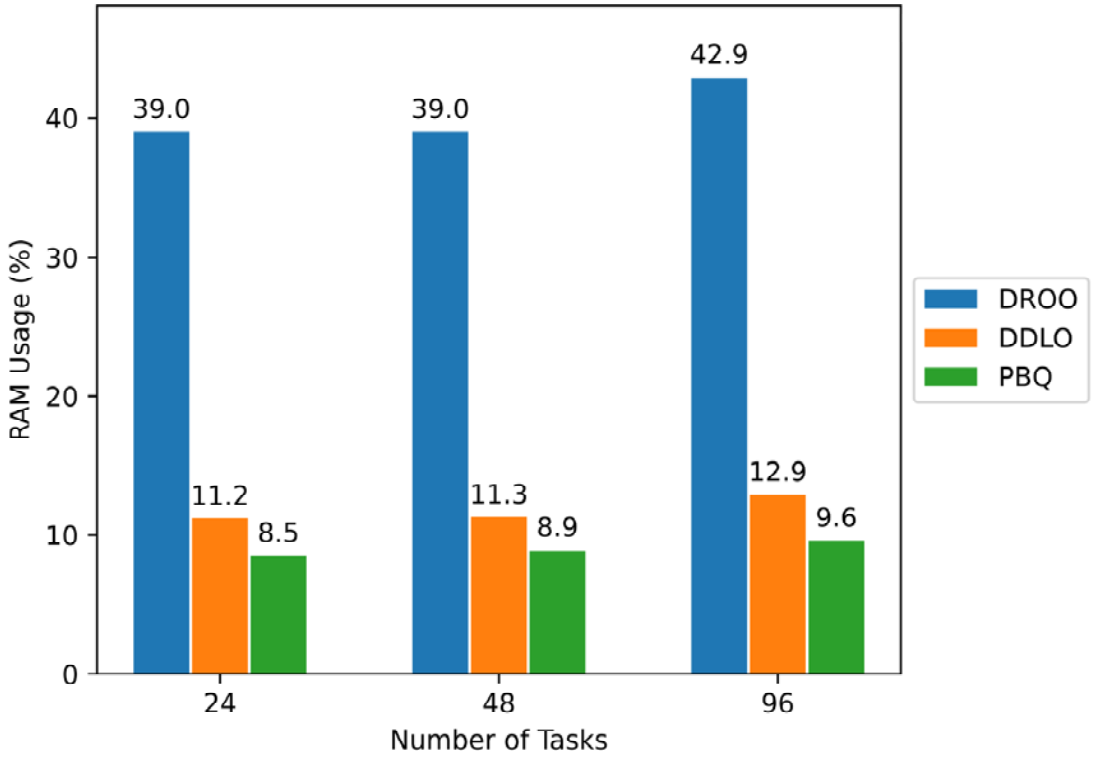
- Computation cost of PBQ is on average
  78% lower than DDLO & DROO in 24 tasks
  65% lower than DDLO & DROO in 48 tasks
  31% lower than DDLO & DROO in 96 tasks.



Experimental results - RAM usage:

- RAM usage is important parameter for IoT devices as they have limited memory resources.

- RAM usage of PBQ is on average
  51% lower than DDLO & DROO in 24 tasks
  48% lower than DDLO & DROO in 48 tasks
  51% lower than DDLO & DROO in 96 tasks.



Conclusion & Future work:

- Introduced a scalable QL based algorithm named PBQ for efficient task scheduling in EC.
- PBQ minimized scheduling time and reduce memory consumption compared to others.
- Implemented a state filtering step to manage memory efficiently by preserving states that meet predefined conditions.
- Developed a priority-based task scheduling approach, prioritizing tasks with the earliest deadlines for accelerated scheduling speed.
- Defined a specialized reward function based on state transition efficiencies, contributing to improved decision-making.
- Comparative analysis with DDLO and DROO scheduling methods revealed that PBQ achieved significant improvements, demonstrating a 72.8% reduction in runtime, 58.1% in computation cost, and 50% in memory usage on average.

References:

[16] Liu, J., Ren, J., Zhang, Y., Peng, X., Zhang, Y., & Yang, Y. (2021). Efficient dependent task offloading for multiple applications in MEC-cloud system. IEEE Transactions on Mobile Computing.

[17] Mahjoubi, A., Taheri, J., Grinnemo, K. J., & Deng, S. (2021, October). Optimal placement of recurrent service chains on distributed edge-cloud infrastructures. In 2021 IEEE 46th Conference on Local Computer Networks (LCN) (pp. 495-502). IEEE.

[19] Cai, L., Wei, X., Xing, C., Zou, X., Zhang, G., & Wang, X. (2021). Failure-resilient DAG task scheduling in edge computing. Computer Networks, 198, 108361.

[20] Liu, Y., Wang, S., Zhao, Q., Du, S., Zhou, A., Ma, X., & Yang, F. (2020). Dependency-aware task scheduling in vehicular edge computing. IEEE Internet of Things Journal, 7(6), 4961-4971.

[21] Chhabra, A., Huang, K. C., Bacanin, N., & Rashid, T. A. (2022). Optimizing bag-of-tasks scheduling on cloud data centers using hybrid swarm-intelligence meta-heuristic. The Journal of Supercomputing, 1-63.

[22] Saravanan, G., Neelakandan, S., Ezhumalai, P., & Maurya, S. (2023). Improved wild horse optimization with levy flight algorithm for effective task scheduling in cloud computing. Journal of Cloud Computing, 12(1), 24.

[24] Gao, Y., Wu, W., Nan, H., Sun, Y., & Si, P. (2020, June). Deep reinforcement learning based task scheduling in mobile blockchain for IoT applications. In ICC 2020-2020 IEEE International Conference on Communications (ICC) (pp. 1-7). IEEE.

[25] Tang, B., Luo, J., Obaidat, M. S., & Vijayakumar, P. (2022). Container-based task scheduling in cloud-edge collaborative environment using priority-aware greedy strategy. Cluster Computing, 1-17.

[26] Zhan, W., Luo, C., Wang, J., Wang, C., Min, G., Duan, H., & Zhu, Q. (2020). Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing. IEEE Internet of Things Journal, 7(6), 5449-5465.

[27] Huang, L., Feng, X., Feng, A., Huang, Y., & Qian, L. P. (2018). Distributed deep learning-based offloading for mobile edge computing networks. Mobile networks and applications, 1-8.

[28] Huang, L., Bi, S., & Zhang, Y. J. A. (2019). Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks. IEEE Transactions on Mobile Computing, 19(11), 2581-2593.